Cloud Composer (Airflow)

Create an Airflow Workflow Using Cloud Composer

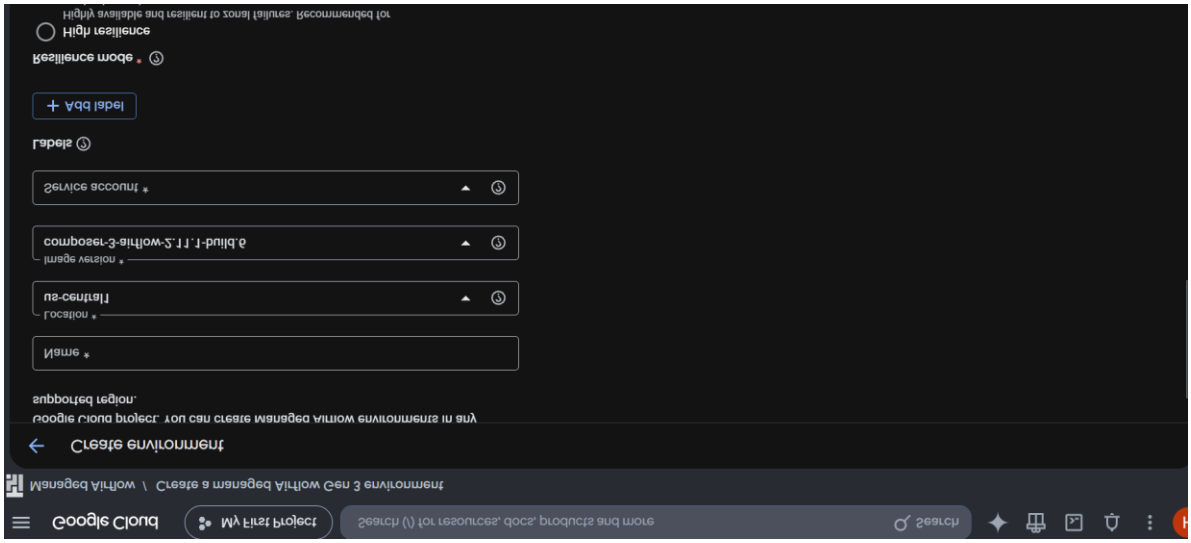
1. Open Google Cloud Console and navigate to Managed AirFlow.
2. Create a Composer environment and wait until the environment status becomes Running.
3. Open the Composer Environment details page.
4. Click Airflow UI to access the Apache Airflow web interface.
5. Open the Composer bucket associated with the environment.
6. Navigate to the **dags** folder inside the Composer bucket.
7. Upload the DAG Python file into the dags folder.
8. Verify that the DAG file is successfully uploaded to Cloud Storage.
9. Open Airflow UI and verify that the DAG appears in the DAG list.
10. Enable the DAG using the toggle button.
11. Trigger the DAG manually using Trigger DAG.
12. Monitor DAG execution from the Graph View and Grid View.
13. Verify that all dependent tasks execute sequentially.
14. Open each task and review execution logs.
15. Confirm successful execution of all workflow stages.
16. Validate the final output and completion status.

DAG with Dependencies
The workflow executes tasks sequentially using task dependencies:

extract_data → validate_data → transform_data → load_data → send_notification

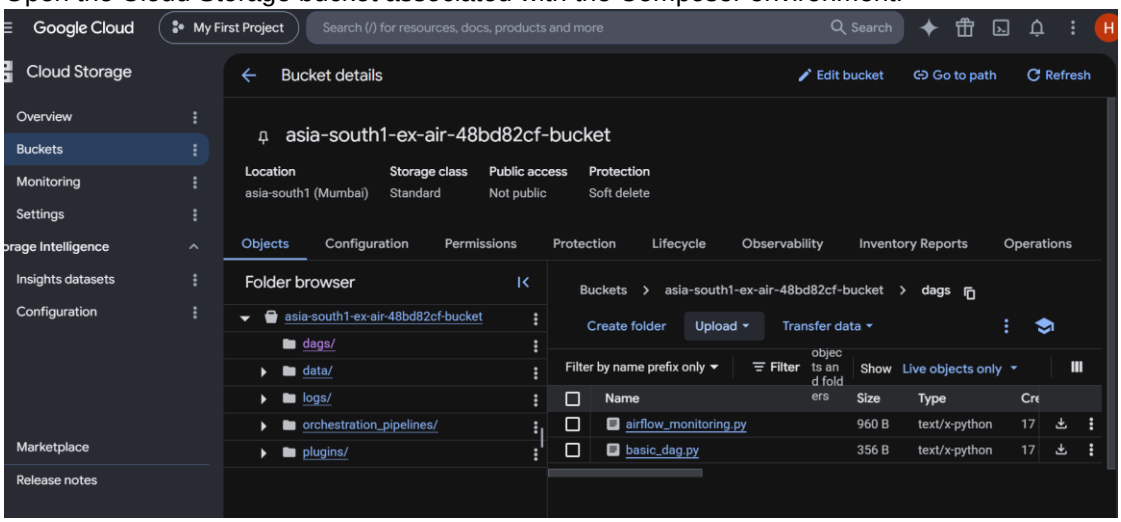
The DAG uses BashOperator tasks and demonstrates workflow orchestration, dependency management, task scheduling, execution monitoring, and logging through Apache Airflow running on Cloud Composer.

Create an Airflow (Cloud Composer) Environment



Open the Composer Bucket

Open the Cloud Storage bucket associated with the Composer environment.



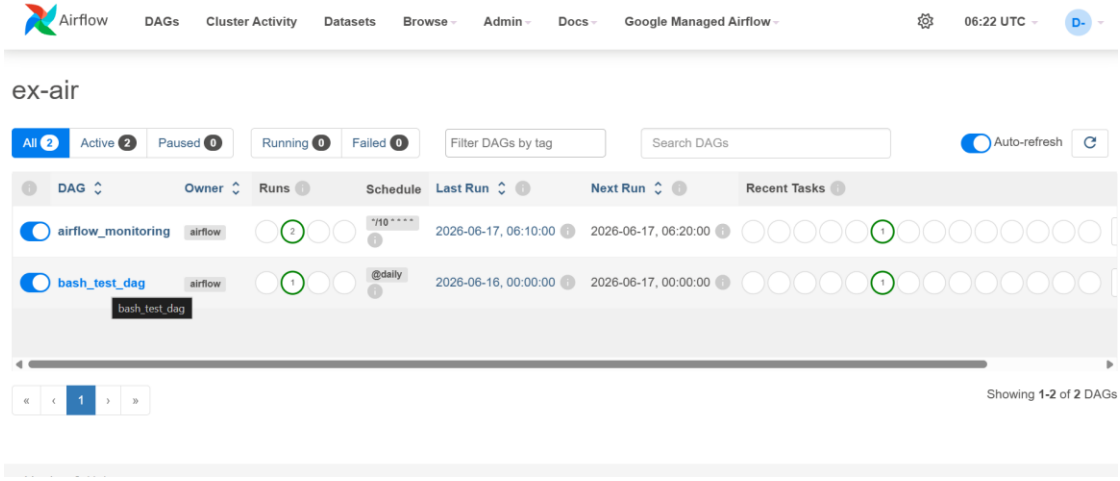
Upload the DAG File

Upload your DAG Python file into the dags folder.

```
: > Users > mahes > OneDrive > Documents > harsha practice > basic_dag.py > ...
1   from airflow import DAG
2   from airflow.operators.bash import BashOperator
3   from datetime import datetime
4
5   with DAG(
6       dag_id="bash_test_dag",
7       start_date=datetime(2025, 1, 1),
8       schedule="@daily",
9       catchup=False,
10  ) as dag:
11
12      hello_task = BashOperator(
13          task_id="hello_task",
14          bash_command='echo "Hello Airflow!"'
15      )
```

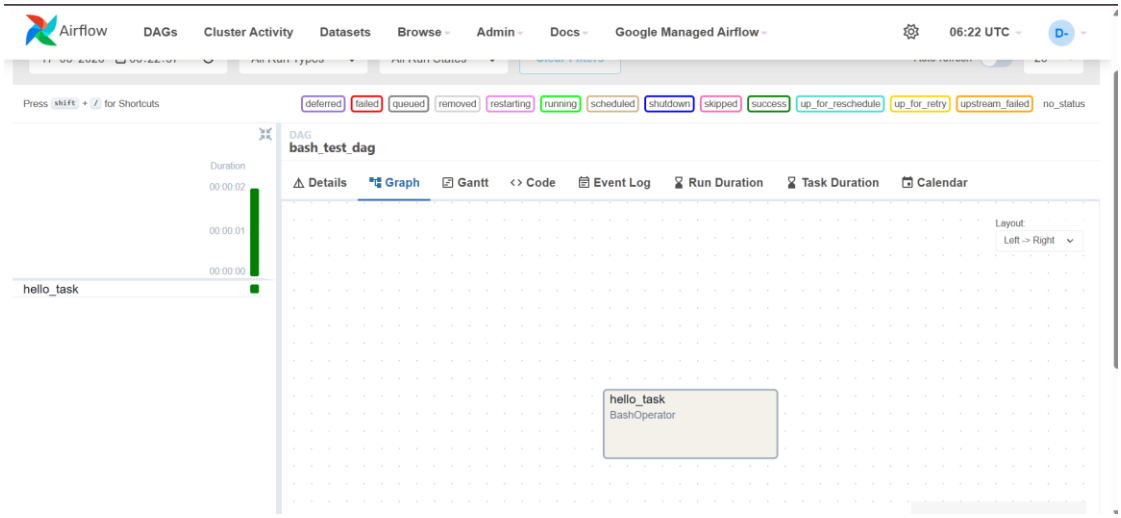
Verify DAG in Airflow UI

Open the Airflow UI and verify that the uploaded DAG appears in the DAG list.



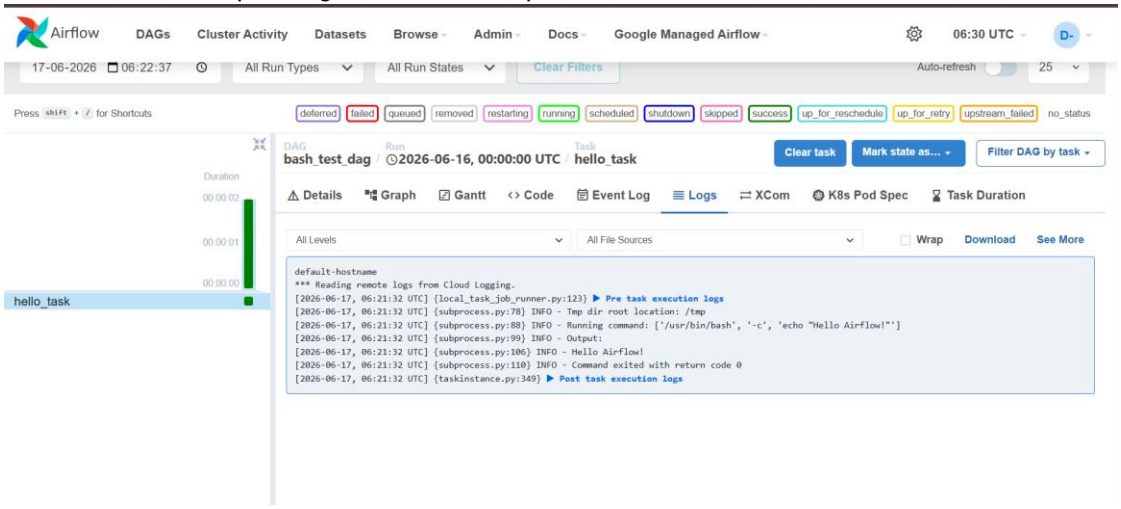
Trigger and Run the DAG

Open the DAG, trigger a run, and monitor task execution.



View Task Output Logs

Click the task and open Logs to view the output such as 'Hello Airflow!'



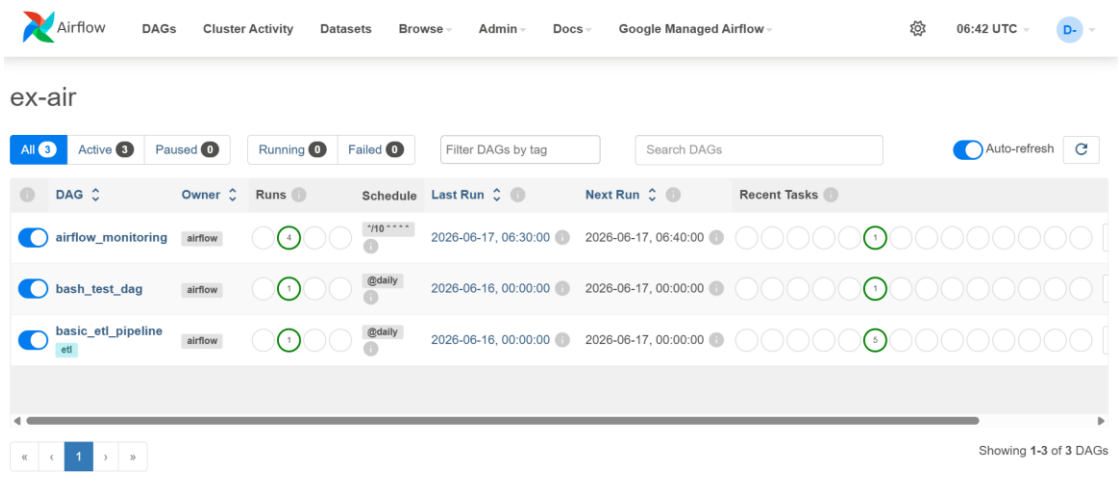
DAG with Dependencies

Create a DAG with task dependencies using BashOperator. The workflow executes tasks sequentially: extract_data → validate_data → transform_data → load_data → send_notification.

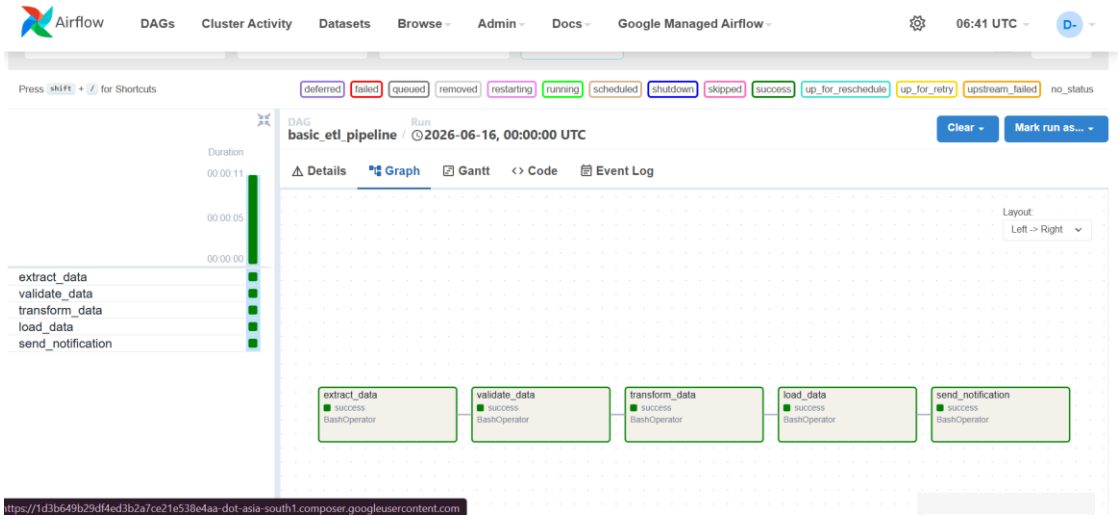
DAG Python Code

```
> Users > mahes > OneDrive > Documents > harsha practice > dag_dep.py > ...
1   from airflow import DAG
2   from airflow.operators.bash import BashOperator
3   from datetime import datetime
4
5   with DAG(
6       dag_id="basic_etl_pipeline",
7       start_date=datetime(2025, 1, 1),
8       schedule="@daily",
9       catchup=False,
10      tags=["etl"],
11  ) as dag:
12
13      extract_data = BashOperator(
14          task_id="extract_data",
15          bash_command='echo "Extracting data from source..."',
16      )
17
18      validate_data = BashOperator(
19          task_id="validate_data",
```

DAG Available in Airflow UI



DAG Execution Graph with Dependencies



Result

The Cloud Composer workflow executed successfully. The DAG was uploaded to the Composer bucket, detected by Airflow, triggered manually, and executed according to the defined task dependencies. All tasks completed successfully and the execution was verified through Airflow Graph View, Task Logs, and DAG Run Status.

DAG Python Code

```
from airflow import DAG from
airflow.operators.bash import BashOperator from
datetime import datetime

with DAG(
dag_id='dag_with_dependencies',
start_date=datetime(2025,1,1),
schedule=None, catchup=False )
as dag:

extract_data = BashOperator(
task_id='extract_data',
bash_command='echo Extracting data'
)

validate_data = BashOperator(
task_id='validate_data',
bash_command='echo Validating data'
)

transform_data = BashOperator(
task_id='transform_data',
```

```
bash_command='echo Transforming data'
)

load_data = BashOperator(
task_id='load_data',
bash_command='echo Loading data'
)

send_notification = BashOperator(
task_id='send_notification',
bash_command='echo Sending notification' ) extract_data >> validate_data >>
transform_data >> load_data >> send_notification
```